

Using AI Tools and Python to Automate Tasks

A two-day, beginner-friendly journey to streamline workflows, boost productivity, and integrate AI into everyday tasks.

Goals

- 1. Understand the basics of Python scripting without prior programming experience**
- 1. Explore ways to leverage LLMs in combination with Python**
- 1. Learn how to automate tasks such as file organization and data entry**
- 1. Use Python scripts to integrate with external LLM APIs**

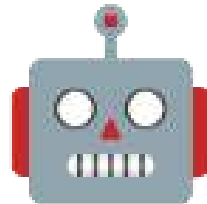
Programming?

Making Machines Do What You Want 🤖

Example: Instructing a Robot

- The Task: **"Move forward, pick up the box, turn right, place the box"**
- The Code:

```
def deliver_package():  
    robot.move_forward(steps=10)  
    robot.pick_up("box")  
    robot.turn_right(90)  
    robot.place_item("box")  
    robot.say("Package delivered!")
```

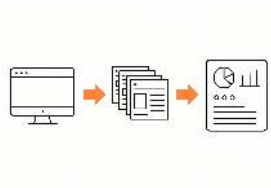


The robot follows your instructions exactly!

It's Like Writing Recipes

```
# Pancake Making Algorithm
def make_pancake():
    # Step 1: Prepare batter base
    batter = mix_flour_and_milk()
    # Step 2: Complete the batter
    batter = add_egg_and_stir(batter)
    # Step 3: Prepare cooking surface
    pan = heat_pan_on_stove()
    # Step 4: Start cooking
    pour_batter_in_pan(pan, batter)
    # Step 5: Wait for indicator
    wait_for_bubbles()
    # Step 6: Turn the pancake
    flip_pancake()
    # Step 7: Finish cooking
    cook_other_side()
    return pancake
```

Why Learn to Automate?



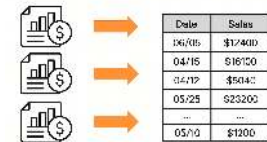
Automate repetitive tasks



Increase productivity



Replace subscriptions (save \$)



Analyse massive amounts of data

Date	Sales
06/05	\$17400
04/15	\$16100
02/12	\$5040
05/25	\$23200
05/10	\$1200

The Real Reason...

Writing code = the most general skill you can have!

It's a superpower :)

Automation is for Everyone

Learning to automate $\neq$ everyone should be a developer

- Automation skills are for **everyone**
- Professional software engineering is a **specialized career**
- You can be productive with code **without becoming a full-time developer**
- **Script** = a simple program that automates a task
- **Professional software** = a complex program engineered for many people

Scripting ≠ 'Software Engineering'

Scripting is:

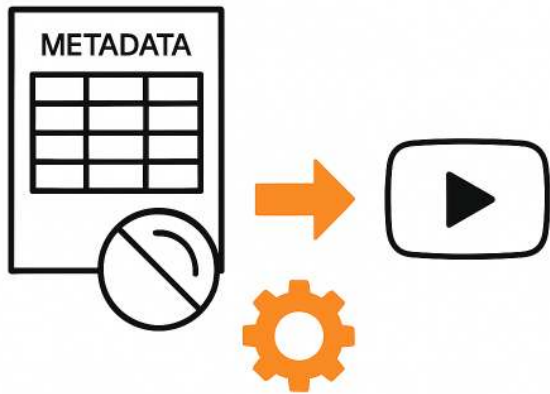
- Writing code to fulfil a set of pre-defined steps that accomplish a task
- Quick, targeted, and task-oriented
- Something you don't overthink
- Typically for automations, data manipulation, or utility tasks

Who Are You Building For?

AUDIENCE	RELIABILITY	SECURITY	SCALABILITY	UX
Public	High	High	High	High
Partners	Medium	Medium	Medium	Medium
Internal	Medium	Low	Low	Medium
Personal	Low	Low	Low	Low

The audience shapes every technical decision you make

Real Automation Examples (that I use!)



Bulk YouTube Upload

50 videos, metadata from a spreadsheet —
saved 10+ hrs



Notes Processor

Scattered notes → organized summaries with
action items

**Your automation doesn't need to be perfect
—it just needs to solve YOUR problem!**

Why Python?

What is Python?



Python is a high-level, general-purpose programming language designed for readability and simplicity.

Created by Guido van Rossum in 1991, Python emphasizes code that reads like English.

English:

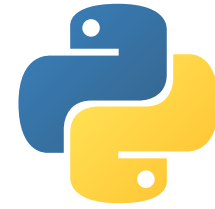
"For each item in my shopping list,
print the item"

Python:

```
for item in shopping_list:  
    print(item)
```

See how similar they are? That's Python's superpower!

Python is Easy and Everywhere



- **General Purpose:** web, data science, automation, AI, and more
- **#1 language** among developers worldwide (TIOBE, Stack Overflow)
- **Powers:** 🤖 AI/ML · 🚗 self-driving cars · 🌐 Instagram/Spotify/Netflix · 🔬 research
- **Beginner-friendly:** reads like English, minimal syntax

Scripting with Python

- Much faster learning curve - *if it works don't change it*
- Immediate practical benefits - *makes your life better*
- Ideal for nondevelopers who just want to get things done
- **It's about learning to solve YOUR specific problems**
- Scripts should be **tailored to your unique needs**, not one-size-fits-all solutions

The Ostrich Approach to Learning Python





WIKIPEDIA
The Free Encyclopedia



[Donate](#) [Create account](#) [Log in](#) ...

≡ Ostrich algorithm

🌐 14 languages ▾

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) ▾

From Wikipedia, the free encyclopedia

In [computer science](#), the **ostrich algorithm** is a strategy of ignoring potential problems on the basis that they may be exceedingly rare. It is named after the [ostrich effect](#) which is defined as "to stick one's head in the sand and pretend there is no problem". It is used when it appears the situation may be more cost-effectively managed by allowing the problem to continue to occur rather than to attempt its prevention.

It's about Learning What We Need for Our Tasks!

And purposefully ignoring that which does not seem to have any effect on its success.

Where we're going

Building intuition before writing code

Part 1 — What Python *can* do for you A framework for spotting automations in your own work.

Part 2 — What Python *can't* do for you A framework for avoiding expensive dead ends.

*The hard part of automation isn't the code. AI can write the code. The hard part is **seeing the automation**.*

Part 1

What can you automate?

The core mental model

Every automation is three things:

INPUT → PROCESS → OUTPUT

BLANK

EXAMPLE

Input?

A folder, CSV, URL, PDF, clipboard, API call

Process?

Rename, filter, extract, summarize, convert

Output?

A new file, a report, an email, a sheet row

Example: "Rename all photos in a folder by the date they were taken." →
Input: a folder of photos · Process: read each EXIF date · Output: renamed files

If you can say it in one sentence, **you have a script.**

The sniff test — is it worth automating?

Four yes/no questions:

1. Do I do this **more than once**?
2. Are the steps **predictable**?
3. Does the input live somewhere a computer can **reach**?
4. Is the output something a computer can **produce**?

4 yeses → strong candidate **2 or fewer** → not worth automating yet

| *Twice is a coincidence. Three times is a script.*

The five categories

Most beginner-friendly automations fall into one of these buckets.

#	CATEGORY	WHAT IT TOUCHES
1	Files & folders	Your filesystem
2	Text & data	Documents, tables, PDFs
3	The web	URLs, pages, public data
4	App integrations	Gmail, Slack, Notion, etc.
5	AI-powered	Language & judgment tasks

A teaching shortcut, not an official taxonomy.

Category examples

1. **Files & folders** — rename 300 photos by date · sort Downloads by type · weekly zip-backup
2. **Text & data** — clean a CSV · extract emails from PDFs · `.docx` → Markdown
3. **The web** — track a stock price · scrape public headlines · call an API → spreadsheet
4. **App integrations** — log unread Gmail · daily Slack summary · sync calendar → Notion
5. **AI-powered** — summarize meeting notes · classify support emails · clean up transcripts

| *Green-zone work: data is **reachable**, rules are **describable**, errors are **cheap**.*

From idea to runnable script

1. **Write the one-sentence spec** — *"Take `<input>`, do `<process>`, produce `<output>`."*
2. **Ask an AI to draft it** (template below)
3. **Run it** — `uv run python scripts/my_tool.py --input ./folder`
4. **Turn it into a CLI tool** — `argparse` + save to `scripts/` + a `--help` line

Write a Python script that:

- Takes `<input>` as input
- Does `<process>`
- Produces `<output>`

Make it runnable from the command line with `argparse`.

Part 2

What Python *can't* do for you

The core frame: the four R's

Before committing to automate something, ask:

QUESTION	WHAT IT'S REALLY ASKING
Reach?	Can my code get to the data and act on the system?
Rules?	Can I describe the logic in plain steps?
Reliability?	Am I OK with occasional mistakes?
Risk?	If it misfires, is the worst case recoverable?

The four answers sort every task into a zone.

Three zones

- **GREEN** — Build it. Python is great at this.
- **YELLOW** — Possible, but fragile. Build it small. Babysit it.
- **RED** — Don't aim Python here. Keep a human in the loop.

Green zone — Python shines

- Moving data between files and formats (CSV ↔ JSON ↔ Excel, PDFs, logs)
- Talking to documented APIs (OpenAI, Anthropic, GitHub, Notion, Gmail)
- Scraping **public, static** web pages
- Running things on a schedule (backups, daily reports)
- LLM language tasks: summarize, classify, extract, rewrite
- Gluing tools together — "when X happens in A, write to B"

Common thread: **reachable, describable, cheap to get wrong.**

Yellow zone — possible, but fragile

- Scraping sites with logins, captchas, or heavy JavaScript
- Automating desktop apps by clicking buttons (breaks on every update)
- Parsing messy/inconsistent documents (scanned PDFs, wild emails)
- Anything that "mostly works" 90% of the time — when the other 10% matters
- Long chains of AI calls without verification steps

| *Rule of thumb: build it small, watch it run, expect to babysit.*

Red zone — wrong shape of problem

- **Reach:** closed systems, 2FA/SSO with no API, mobile-only apps, physical tasks
- **Rules:** tasks that need genuine human judgment every time
- **Reliability:** near-perfect is required (medical, legal, safety)
- **Risk:** irreversible or expensive failure (moving money, mass email, bulk delete)

*If any of the four R's lands in red, **keep the human in the loop** — or pick a smaller version of the task.*

The AI-specific trap

Because this course uses AI heavily, two failure modes matter:

1. Silent wrongness LLMs produce confident, fluent output even when it's wrong.

2. Drift The same prompt can give different outputs on different days or model versions.

Habit: print the AI's output before you act on it — for the first dozen runs at least.

30-second "which zone?" checklist

Before you build:

- ☐ I can **reach** the input without manual steps
- ☐ I can **describe the rules** in a paragraph
- ☐ I'm OK with occasional errors, or I have a way to catch them
- ☐ If the script misfires, the worst case is **easy to undo**

4 checks → build it **3 checks** → build small, watch it run **2 or fewer** → don't automate yet. Do it by hand and learn the shape first.

5-minute exercise

1. Write down **three repetitive tasks** from your week
2. For each: fill in **Input** → **Process** → **Output**
3. For each: answer the **four R's** (reach, rules, reliability, risk)
4. Sort them: **green** / **yellow** / **red**
5. Build only the **green** ones this week
6. Write one sentence for each **red** one explaining *why* it's red

That last sentence is often the most valuable thing you'll learn.

The real skill isn't Python.

It's instantly recognizing what's automatable and what isn't — and not wasting a weekend on a task that was always in the red zone.

**Survey Question: Do you have a subscription
for ChatGPT/Claude/Gemini?**

The Busy Person Guide to Python Basics 🐍

Just enough Python to be dangerous.

What We'll Learn

We cover only what's needed to **automate**, and nothing more.

Syntax: indentation, variables, printing

Data types: text, numbers, true / false

Control flow: if, for, while

Libraries: os, csv, requests

A framework: *when* and *how* to automate

Intuition: a feel for what's worth the effort

Demo - Setting Up Your Environment

(Installing Python, uv, Editor, Jupyter Notebook, AI chatbot)



What You Need Installed

TOOL	PURPOSE
Python	the language itself
uv	fast package manager
VS Code	your editor
Jupyter	<code>uv run --with jupyter jupyter lab</code>
An AI chatbot	ChatGPT, Claude, or Gemini

Everything runs through `uv run`, so there's no manual environment activation.

Code Is Mostly Moving Data Around

Nearly every program does one thing: **take data and change it.**

Three words carry the whole idea:

Data: the *stuff* itself (numbers, text, true / false)

Operations: what you *do* to data, like `5 + 5`

Variables: the *names* you give data, like `name = "Lucas"`, so you can reuse it later

Read Code Like a Sentence

```
name = "Lucas"           # a string (text)
actual_age = 33           # an integer
mental_age = 12           # an integer

average = (actual_age + mental_age) / 2  # an operation

print(average)           # show the result -> 22.5 (a float)
```

Each line **names** a piece of data, then **does something** with it.

Data Types; Operations; Variables - Demo

The Building Blocks

Data types

TYPE	EXAMPLE
int	1, 2, 3
float	1.5, 3.14
str	"Hello"
bool	True / False

Operations

Arithmetic: + - * / ** // %

Concatenate: "Lucas" + " rocks"

Logical: and or not

Compare: > < == !=

Variables Are Named Values

Store a value once, then reach for it by name as often as you like.

```
a = 10  
b = 20  
print(a + b)    # 30
```

Demo: Functions, Lists & Loops

Functions

A function bundles steps that do **one job**, so you write them once.

Without one, you repeat yourself:

```
tax = price * tax_rate  
total = price + tax
```

| *...and you would write it again for every new price and tax rate.*

Define Once, Reuse Anywhere

```
def calculate_total(price, tax_rate):  
    tax = price * tax_rate  
    return price + tax
```

Same logic, different inputs:

```
calculate_total(10, 0.05)    # 10.5  
calculate_total(20, 0.10)    # 22.0
```

Anatomy of a Function

```
def greet(name):           # a name and its parameters
    return f"Hi, {name}!"  # hand a value back
```

- **Why bother:** reuse logic, and split big tasks into small ones
- **Rule of thumb:** short, well-named, one job each

Lists

An **ordered collection** of items.

```
tasks = ["Buy groceries", "Finish project", "Call the bank"]
```

Index: reach one item (counting starts at 0)

```
tasks[0] # "Buy groceries"  
tasks[1] # "Finish project"
```

Slice: reach a range

```
tasks[0:2] # ['Buy groceries', 'Finish project']
```

Loops

Run the **same step on every item**, instead of copy-pasting.

```
for task in tasks:  
    print(task)
```

```
for task in tasks:  
    ask_robot_to_do(task)
```

Renaming files in a folder, or processing rows in a table: the same shape every time.

Demo - Dictionaries, Tabular Data, Conditionals

Dictionaries

Store data as **key** / **value** pairs, then look it up by name.

```
prices = {"apple": 0.50, "banana": 0.75, "orange": 0.60}
```

```
prices["apple"]    # 0.50  
prices["banana"]   # 0.75
```

Configs, lookups, caching: anywhere you want the value for a given key.

Comparisons Answer True or False

```
a, b = 10, 20
```

```
a == b    # False  
a != b    # True  
a < b     # True  
a >= b    # False
```

Combine them with logic:

```
True and False    # False  
True or False     # True  
not True          # False
```


Conditionals Make Decisions

`if / elif / else` lets code choose a path based on the data.

```
if a > b:  
    print("a is bigger")  
elif a == b:  
    print("they're equal")  
else:  
    print("b is bigger")
```

Reacting to data is the core of almost every automation.

Poll Questions

Poll Question 1

What will this code output?

```
age = 25
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Answer: Adult

Explanation: Since 25 is greater than or equal to 18, the condition is True, so "Adult" is printed.

Poll Question 2

What's wrong with this code?

```
tasks = ["Email", "Meeting", "Code"]  
print(tasks[3])
```

Answer: Index out of range error!

Explanation: Lists are zero-indexed. This list has indices 0, 1, 2. There is no index 3!

- `tasks[0]` → "Email"
- `tasks[1]` → "Meeting"
- `tasks[2]` → "Code"

Poll Question 3

Fill in the blank to print each fruit:

```
fruits = ["apple", "banana", "cherry"]  
____ fruit ____ fruits:  
    print(fruit)
```

Answer:

```
for fruit in fruits:  
    print(fruit)
```

Explanation: The `for` loop iterates through each item in the list, and `in` checks membership.

Poll Question 4

What does this code print?

```
def greet(name):  
    return "Hello, " + name + "  
  
print(greet("Lucas"))
```

Answer: Hello, Lucas!

Explanation: `greet` is *defined* with `def`, then *called* with `greet("Lucas")`. The argument `"Lucas"` fills the `name` parameter, and the function `return`s the assembled string, which `print` displays.

Poll Question 5

What does this loop print?

```
numbers = [2, 4, 6]
total = 0
for n in numbers:
    total = total + n
print(total)
```

Answer: 12

Explanation: The `for` loop visits each item in the list once, adding it to `total` (2, then 6, then 12). After the loop finishes, `total` holds the sum of every element.

Poll Question 6

What does this code print?

```
name = "Lucas"  
age = 33  
print(f"{name} is {age} years old")
```

Answer: Lucas is 33 years old

Explanation: An **f-string** (the **f** before the quotes) lets you drop variables directly inside **{ }**. Python replaces **{name}** and **{age}** with their values — no **+** or manual conversion needed.

Poll Question 7

What is this code doing?

```
from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model="gpt-5.6-luna",
    input="Say hello in one word."
)
print(response.output_text)
```

Answer: Sending a prompt to an AI model and printing its reply.

Explanation: We create a `client`, send our `input` (the prompt) to a `model`, and the AI sends back a `response`. This is the core input → process → output pattern for every AI API call.

Poll Question 8

What's the safest way to read a file in Python?

```
with open("notes.txt", "r") as f:  
    contents = f.read()  
print(contents)
```

Answer: Using `with open(...)` — it closes the file automatically.

Explanation: The `with` block opens the file, lets you `read()` it, then **closes it for you** even if something goes wrong. `"r"` means read mode; use `"w"` to write and `"a"` to append.

Poll Question 9

 *Type your answer in the chat before the reveal.*

What does this print?

```
price = 19.5  
print(f"Total: ${price:.2f}")
```

Answer: Total: \$19.50

Explanation: Inside an f-string, `:.2f` formats the number with exactly two decimal places — perfect for money.

Poll Question 10

 *Type your answer in the chat before the reveal.*

What does this print?

```
contacts = {"Alice": "555-0101"}  
print(contacts.get("Bob", "Not found"))
```

Answer: Not found

Explanation: `.get(key, default)` returns the default when the key is missing — so it won't crash like `contacts["Bob"]` would.

Poll Question 11

 *Type your answer in the chat before the reveal.*

What does `tasks[1:3]` give you?

```
tasks = ["email", "meeting", "code", "lunch"]  
print(tasks[1:3])
```

Answer: `['meeting', 'code']`

Explanation: Slicing `[1:3]` takes items at index 1 and 2 — the start is included, the end (3) is **not**.

Poll Question 12

 *Type your answer in the chat before the reveal.*

What's wrong with this code?

```
age = input("Your age: ")  
print(age + 10)
```

Answer: `input()` returns a **string**, so `"30" + 10` is a `TypeError`.

Explanation: Wrap it in `int(...)`: `age = int(input("Your age: "))`. Then the math works.

Poll Question 13

 *Type your answer in the chat before the reveal.*

Fill in the blanks to print the grade:

```
score = 75
— score >= 90:
  print("A")
— score >= 70:
  print("B")
—:
  print("C")
```

Poll Question 13 — Answer

Answer:

```
if score >= 90:  
    print("A")  
elif score >= 70:  
    print("B")  
else:  
    print("C")
```

Explanation: Python checks `if` → `elif` → `else` top to bottom and runs the **first** branch that's true. Here `score = 75` prints `B`.

Poll Question 14

 *Type your answer in the chat before the reveal.*

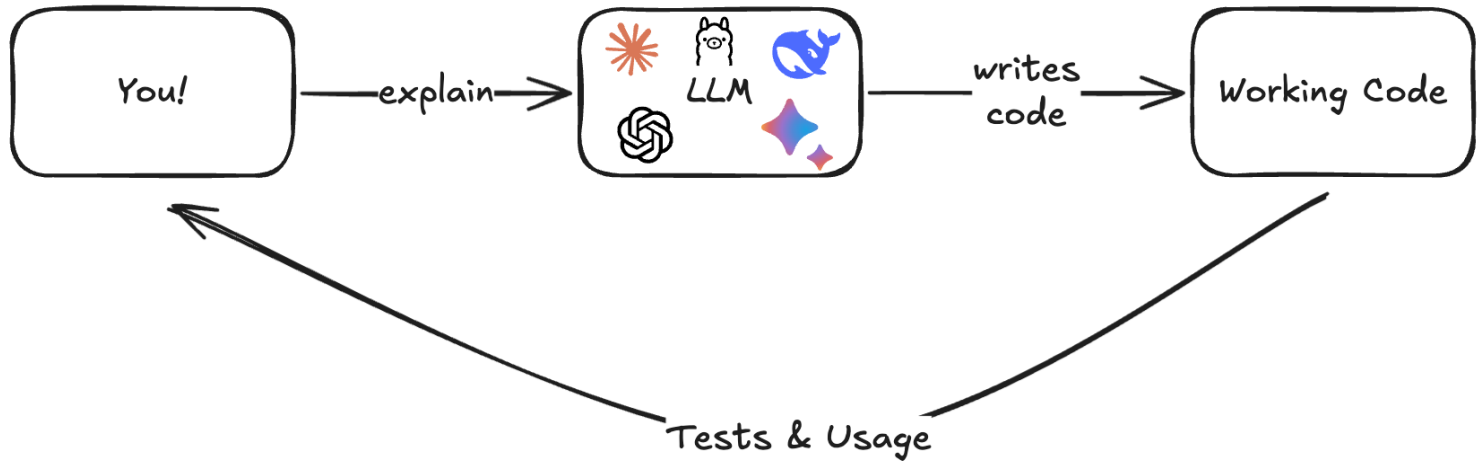
What is this code doing?

```
for n in [10, 7, 4, 3]:  
    if n % 2 == 0:  
        print(n, "is even")
```

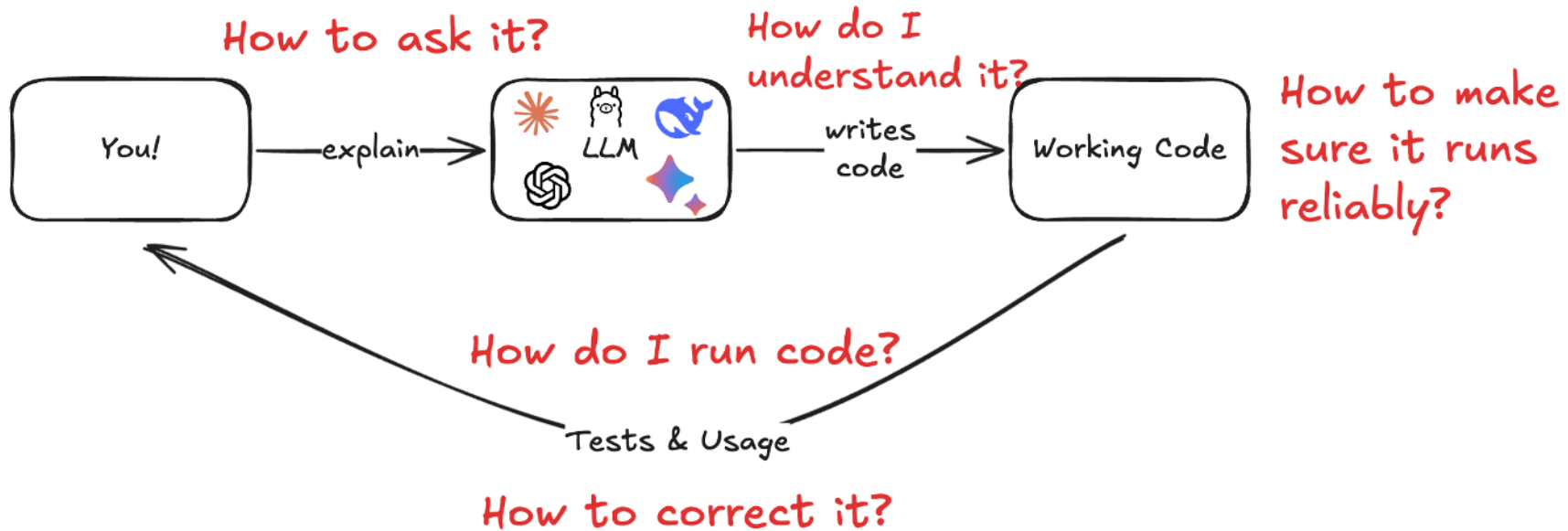
Answer: Printing the even numbers: 10 is even, then 4 is even.

Explanation: $n \% 2$ is the remainder after dividing by 2. When it's 0, the number is even — a super common automation check.

The "Just Ask AI" Naive Approach



The "Just Ask AI" Naive Approach



Demo - Working with Data, APIs & Packages

Reading/Writing

- **Tabular Data:** Data that is organized in a table format
- **Common Formats:** `.csv`, `.json`, `.xlsx`, `.xls`
- CSV Files store data in rows and columns via comma-separated values:

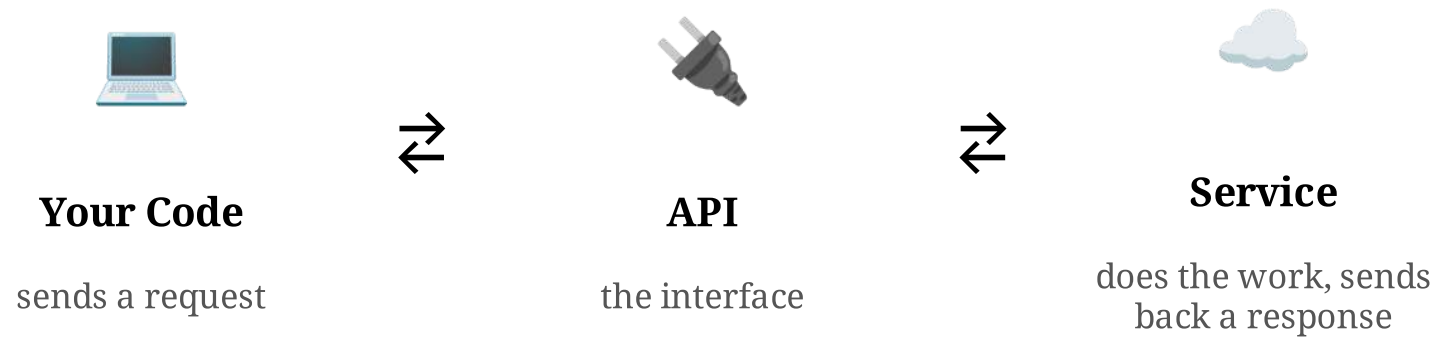
```
Name,Age,City  
John Smith,32,New York  
Jane Doe,28,San Francisco
```

NAME	AGE	CITY
John Smith	32	New York
Jane Doe	28	San Francisco

Understanding APIs & Packages

What is an API?

API = Application Programming Interface — a way for programs to talk to each other.



APIs in the Real World

Neither Booking.com nor Airbnb built their own maps — they just call Google's API.



Booking.com

Shows hotel locations on map



Google Maps API



Airbnb

Shows property locations

What are Packages and Imports?

Package = A collection of pre-written Python code (functions, classes) that you can reuse

Import = The command to bring a package into your script so you can use it

Example:

```
# Import the 'requests' package to make HTTP calls
import requests

# Now we can use functions from the requests package
response = requests.get("https://api.example.com/data")
print(response.json())
```

What are Packages and Imports? (cont.)

Think of it like:

- Package = A toolbox full of tools
- Import = Taking the toolbox out so you can use the tools
- Using the package = Actually using the tools to build something

Popular packages: `requests` (HTTP calls), `pandas` (data analysis), `openai` (AI services)

Package Versions: What Does `x . x . x` Mean?

Every package has a version number with **3 parts** — e.g., `requests==2.31.0`

NUMBER	NAME	CHANGES WHEN...
2.31.0	Major	Breaking changes — old code may stop working
2.31.0	Minor	New features added, backwards compatible
2.31.0	Patch	Bug fixes only, safe to update

Package Versions: What Does `x . x . x` Mean? (cont.)

Rule of thumb:

- Patch bumps → safe to update anytime
- Minor bumps → usually safe, check changelog
- Major bumps → read the migration guide ⚠

```
# Pinning exact versions = reproducible, predictable code
requests==2.31.0    # this exact version
requests>=2.31.0   # this version or newer
requests~=2.31.0   # 2.31.x only (patch updates ok)
```

The Package Management Problem

When you write a Python automation script, it needs:

1. **A specific Python version** (e.g., Python 3.11)
1. **Specific package versions** (e.g., `requests==2.31.0`, `pandas==2.0.1`)
1. **Isolated environment** (so different projects don't conflict)

The Package Management Problem (cont.)



Traditional Approach: Multiple Files + Virtual Environments

 **automation.py**

```
import requests
import pandas

# Your code here...
```

 **requirements.txt**

```
requests=2.31.0
pandas=2.0.1
numpy=1.24.0
```

 **.python-version**

```
3.11
```

Tools: pipenv, conda, venv, poetry... **Confusing for beginners!**

Demo - The Package Management Problem

Enter UV: The Simple Solution ✨

UV = Python script runner + package manager in one

Everything in a single file!

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "requests==2.31.0",
#     "pandas==2.0.1",
# ]
# ///

import requests
import pandas as pd

# Your automation code here...
response = requests.get("https://api.example.com/data")
df = pd.DataFrame(response.json())
print(df.head())
```


UV: Single File vs Traditional Approach

Traditional 🥵

- Create virtual environment
- Activate environment
- Install packages
- Manage requirements.txt
- Remember which env to use

📁 Multiple files · 📖 Complex workflow

UV 🚀

- Write your script
- Add dependencies at the top
- Run: `uv run script.py`

📄 One file · ⚡ Simple workflow

Perfect for vibe scripting! → Learn more: docs.astral.sh/uv/guides/scripts

What is an Alias?

An **alias** is a shortcut name for a longer command.

Instead of typing:

```
uv run ~/scripts/my_script.py
```

You type:

```
myscript
```

You define it once — then it works every time you open a terminal.

Add an Alias — One Command

Mac / Linux — paste in Terminal:

```
echo "alias myscript='uv run ~/scripts/my_script.py'" >> ~/.zshrc && source ~
```

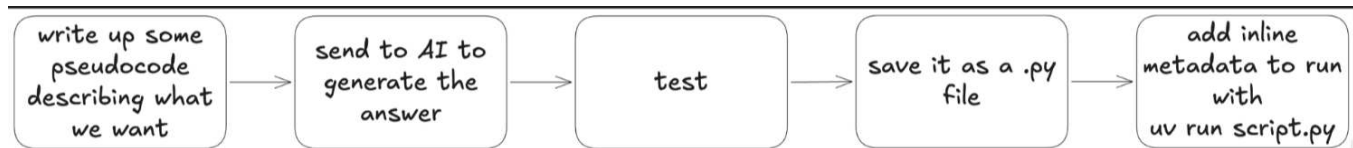
Windows — paste in PowerShell:

```
Add-Content $PROFILE "`nfunction myscript { uv run C:\scripts\my_script.py @a
```

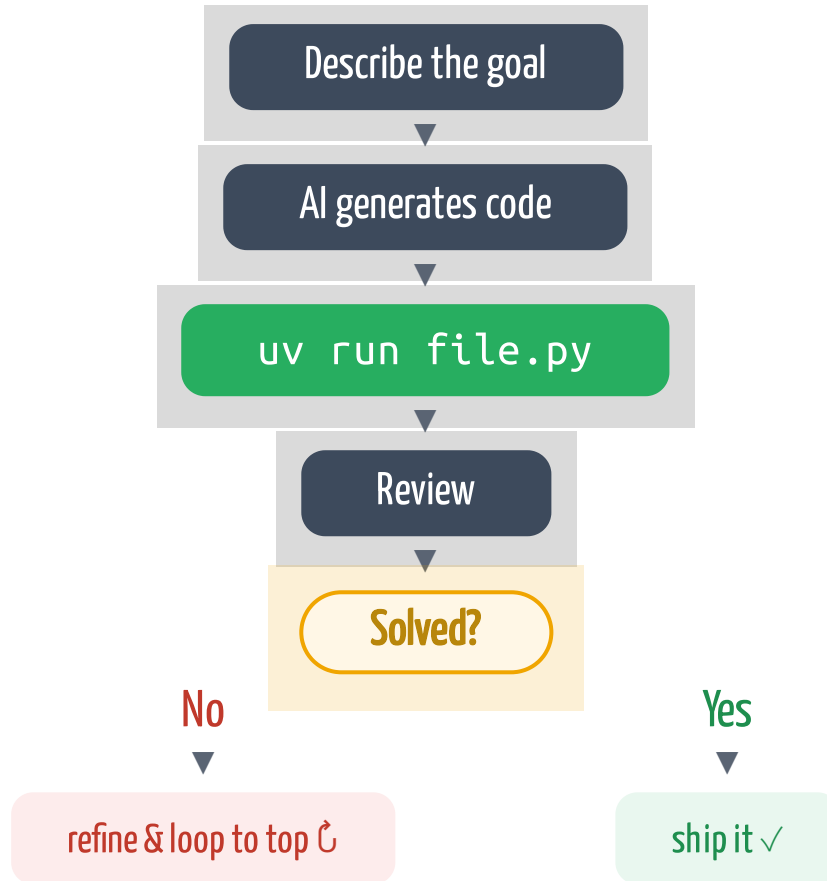
Replace `myscript` with the name you want, and the path with your script's path.

Demo - Vibe Scripting with UV + AI

The Vibe Scripting Steps



The Vibe Scripting Workflow



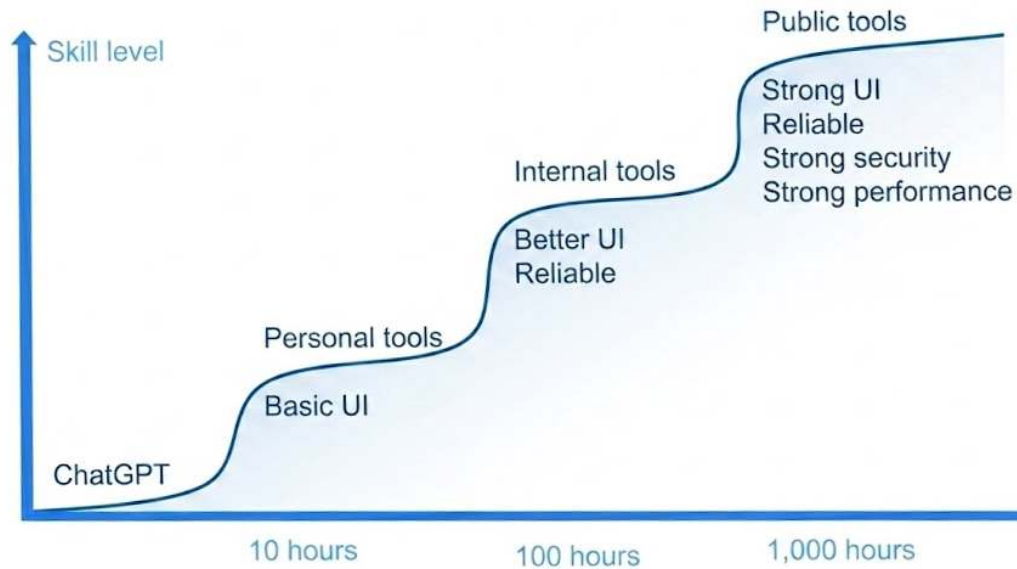
How to Use AI to Learn/Use Python

A quick detour to set up an AI toolkit to speed up our Automation skills

Learning Python with AI

Your Personal Programming Tutor

The AI Coding Upskilling Curve



Demo - Conversational Learning with AI for Python

File Management Automation

Your Downloads folder has 2,000 files and you can't find anything. Python can sort, rename, and tidy all of it in seconds.

- **Organise a messy folder** by file type — zero manual effort
- **Categorise images** using AI vision — no manual sorting needed
- **Analyse a directory** by file type and generate a visual summary

```
`scripts/demos/file-management`
```

PDF & Document Automation

You have 50 invoices to read and a spreadsheet to fill by hand. AI can pull every number out for you while you grab a coffee.

- **Summarise any PDF** in one command — research papers, reports, anything
- **Extract structured data** from invoices and receipts with AI — straight into a spreadsheet
- **Chat interactively** with any PDF — ask questions, get answers

``scripts/demos/pdf-documents``

Data & Dashboards

You rebuild the same Excel chart every Monday morning. Python can fetch the data, refresh the charts, and even explain the trends for you.

- **Build interactive dashboards** from any CSV in minutes with plotly
- **Fetch live API data** and persist it automatically in a local database
- **Analyse log files with AI** — spot patterns and get charts automatically

```
`scripts/demos/data-dashboards`
```

Email & Comms Automation

You open 80 unread emails every morning trying to spot the urgent ones. AI can read them all and hand you a two-line triage list.

- **Summarise daily news feeds** into a briefing with AI — never miss what matters
- **Filter emails by urgency** using AI — triage your inbox automatically
- **Summarise user research notes** into structured insights with AI

``scripts/demos/email-comms``

Demo - Prompts, Tips & Tricks

Using AI != Slop

Using AI != Slop



Watching in real time as "slop" becomes a term of art. the way that "spam" became the term for unwanted emails, "slop" is going in the dictionary as the term for unwanted AI generated content



gabe @allgarbled · May 4

it's cool how every google search now starts with a wall of LLM slop that is completely useless and takes up half the screen



airpods pro 2nd generation



All

Shopping

Images

Videos

Forums

News

Maps



AI Answer

Learn more



AirPods Pro Apple are wireless earbuds that were

Slop comes from vague prompts and zero review — not from using AI

Using AI != Slop

1. **Slop** comes from vague prompts; **craft** comes from specific, **capability-aware prompts**. Name the libraries, the input, and the exact behavior:

"Write me a Python CLI tool using Click and boto3 which takes a URL of that form and then uses EVERY single boto3 trick in the book to try and debug why the file is returning a 404." — Simon Willison

Using AI != Slop (cont.)

1. **Encode your conventions once** — reusable context beats one-off prompting. Save your standards in a Claude Project / system prompt so every output inherits them.
1. **Ship self-contained, runnable artifacts** — the same `uv run` + inline-dependency pattern this course already teaches means anyone can run your script with zero setup.
1. **You own what you run.** As Willison puts it, others can run your script *"provided you trust me not to have replaced the script with something malicious."* AI output is a draft **you** are accountable for.

Red → Green: Tell the AI Exactly What "Done" Means

Write the check **first**, then let AI fill in the code.

- **RED**: write a test that describes the result you want — run it, watch it fail.
- **GREEN**: ask the AI to write code until the test passes.
- **Why**: a failing test first is a clear instruction the AI can't fudge — it stops the AI from "writing code that doesn't work, or ... unnecessary ... or both."

A "test" is just a tiny piece of code that says: if my function works, this should be true.

A 6-Line Example Beginners Can Read

No framework needed — just a plain `assert`:

```
# RED: describe what "done" looks like
# (this fails – get_headers doesn't exist yet)
def test_get_headers():
    md = "# Title\n\nsome text\n## Section"
    assert get_headers(md) == ["Title", "Section"]

test_get_headers() # run it -> error = RED
```

Then the prompt to the AI:

*"Build a Python function `get_headers(markdown)` that returns the header text.
Use red/green TDD."*

Once `test_get_headers()` runs with **no error**, you're **GREEN** — and you keep the test forever as a safety net. 

Connect With Me



[Course materials](#)



[LinkedIn](#)



[Twitter/X - @LucasEnkrateia](#)



[YouTube - @automatalearninglab](#)



Email: lucasenkrateia@gmail.com